

Teoria di *Algoritmi e strutture di dati*

Nicolò Giuliani

Analisi del costo Computazionale

Attraverso il **Master Theorem** risolviamo ricorrenze del tipo $T(n) = \begin{cases} 1 & n = 1 \\ T(n) = aT(\frac{n}{b}) + cn^\beta & n > 1 \end{cases}$:

$$\alpha = \frac{\log a}{\log b}$$

- Se $\alpha > \beta$ allora la complessita' sara' $\Theta(n^\alpha)$
- Se $\beta > \alpha$ allora la complessita' sara' $\Theta(n^\beta)$
- Se $\alpha = \beta$ allora la complessita' sara' $n^\alpha \log n$

Nota bene: con log si intende sempre $\log_2$.

Ricordiamoci di una utile sommatoria notevole:

$$\sum_{i=0}^n i = \frac{n(n+1)}{2}$$

- Costo pessimo: O
- Costo ottimo: Ω
- Costo medio: Θ

1 Liste

Una lista e' una struttura di dati in cui gli elementi sono ordinati in ordine sequenziale.

```
struct list{  
    int value;  
    list* next;  
};
```

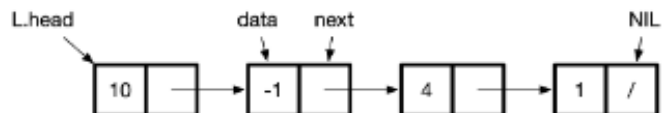


Figure 1: Struttura di una lista singola

Esiste anche un'altra variante ovvero: Lista doppiamente concatenata

```
struct double_linked_list{  
    int value;  
    list* next;  
    list* prev;  
};
```

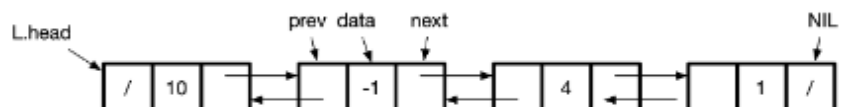
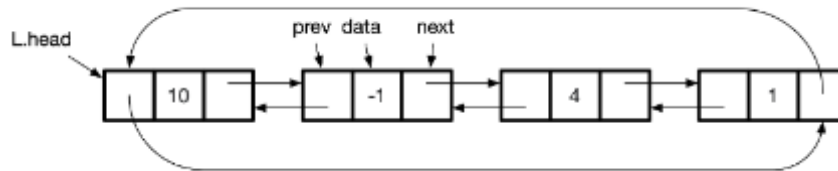


Figure 2: Struttura di una lista doppia

Possiamo trovare anche liste circolari, ovvero che alla fine come puntatore ritorna l'indirizzo del head.



2 Alberi binari

Un albero è una struttura di dati non-lineare gerarchica.

```
struct tree {
    int value;
    tree* left;
    tree* right;

    // Costruttore
    tree(int val) : value(val), left(nullptr), right(nullptr) {}
};
```

Figure 3: Struttura di un albero

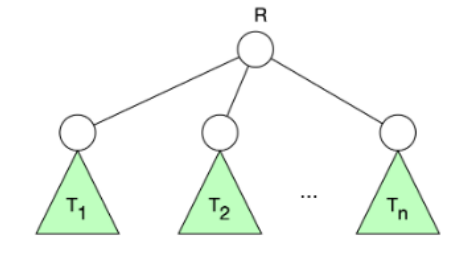


Figure 4: Esempio di albero

In un albero binario ogni nodo può avere massimo due figli, si definisce *foglia* il nodo che non ha figli.

- **Profondità:** è la lunghezza di un percorso, unico, che va dalla radice fino ad un nodo specifico, e' sempre $\log n$ (con n come foglie totali).
- **Livello:** è l'insieme di tutti i nodi ad una profondità'.
- **Altezza:** è la massima profondità', un albero vuoto ha come altezza -1 .
- **Grado:** numero di figli di un nodo.

Si definisce albero binario **completo** se ogni nodo con figli ne ha 2, mentre i vuoti ne hanno 0.

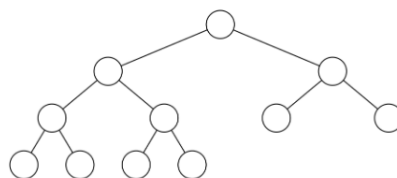


Figure 5: Esempio di albero completo

Si definisce albero binario **perfetto** se l'albero è completo e simmetrico.

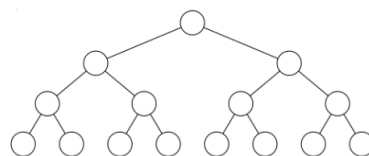
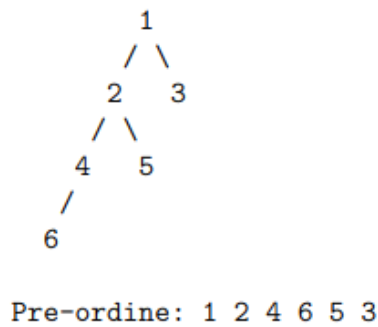


Figure 6: Esempio di albero perfetto

2.1 Visita di profondità

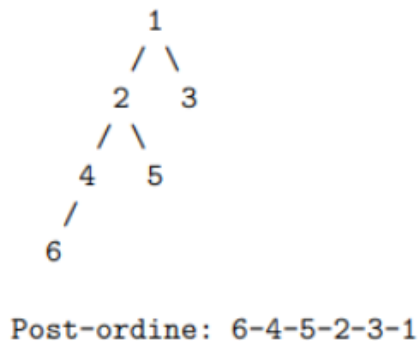
- **Pre-ordine:** stampa prima padre, figlio sinistro e poi destro.



```
void visit(tree* T){
    std::cout << T->value << " ";
}
void pre_order(tree* T){
    if(T){
        visit(T);
        pre_order(T->left);
        pre_order(T->right);
    }
}
```

Costo (ottimo, pessimo, medio): $\Theta(n)$

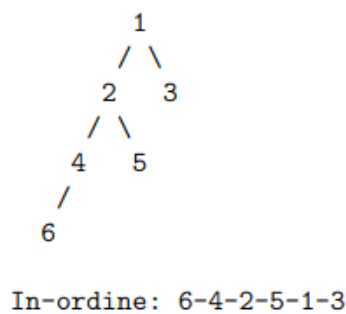
- **Post-ordine:** stampa prima figlio sinistro, destro, padre.



```
void post_order(tree* T){
    if(T){
        post_order(T->left);
        post_order(T->right);
        visit(T);
    }
}
```

Costo (ottimo, pessimo, medio): $\Theta(n)$

- **In-ordine:** stampa prima figlio sinistro, padre, destro.

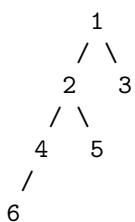


```
void in_order(tree* T){
    if(T){
        in_order(T->left);
        visit(T);
        in_order(T->right);
    }
}
```

Costo (ottimo, pessimo, medio): $\Theta(n)$

2.2 Visita in ampiezza (livelli)

Stampa prima padre, poi figlio sinistro, destro e poi sottofigli.



In-ordine: 1-2-3-4-5-6

```

struct queue { ...
};
void BFS(tree* T){
    queue* Q = new queue();
    if(T){
        Q->enqueue(T); //metti in prima pos della cosa la radice
    }
    while(Q->size != 0){
        tree* x = Q->dequeue(); //togli e prendi il primo elemento della coda
        visit(x); //stampa quel elemento
        if(x->left){
            Q->enqueue(x->left); //metti in coda il figlio sinistro
        }
        if(x->right){
            Q->enqueue(x->right); //metti in coda il figlio destro
        }
        //continua finche' ci sono figli
    }
}

```

Costo (ottimo, pessimo, medio): $\Theta(n)$

2.3 Contare i nodi

```

int countnodes(tree* T){
    if (!T) return 0;
    else return 1+countnodes(T->left)+countnodes(T->right);
}

```

Costo (ottimo, pessimo, medio): $\Theta(n)$

Nota: n indica sempre la profondita' dell'albero.

3 Alberi n-ario

Ovvero un albero con un padre e come figlio una lista di nodi.

```
struct NTree {
    int value;
    NTree* firstChild; // Puntatore al primo figlio
    NTree* nextSibling; // Puntatore al fratello successivo
    NTree* father;

    // Costruttore
    NTree(int val) : value(val), firstChild(nullptr), nextSibling(nullptr), father(nullptr)
};
```

- Ogni nodo ha un puntatore al primo (first) figlio
- Ogni nodo ha un puntatore al fratello successivo (next)
- La lista di figli e' gestita con una lista concatenata semplice

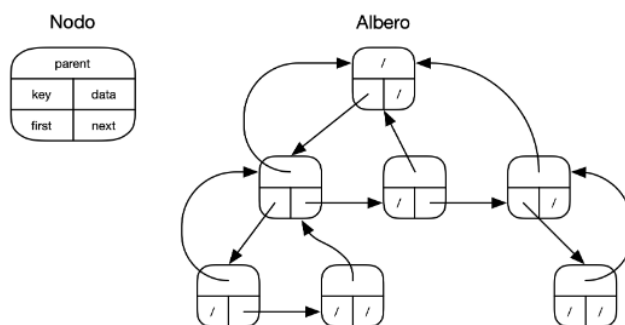


Figure 7: Esempio di albero n-ario

Esiste anche un'altra versione piu' semplice dove abbiamo un padre e un array di figli di dimensione fissa.

```
#define K 10
struct NTree {
    int value;
    NTree* child[K];

    // Costruttore
    NTree(int val) : value(val) { ...
}
};
```

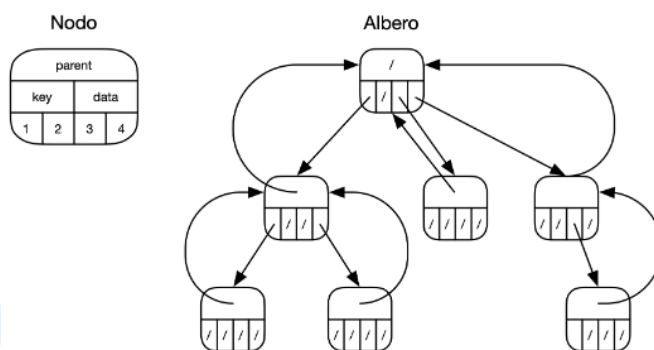
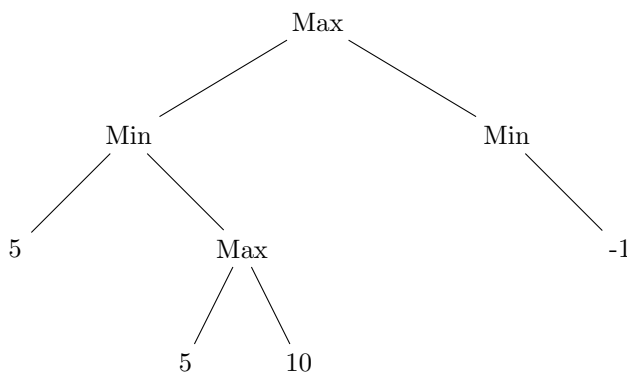


Figure 8: Seconda tipologia di albero n-ario

Minimax

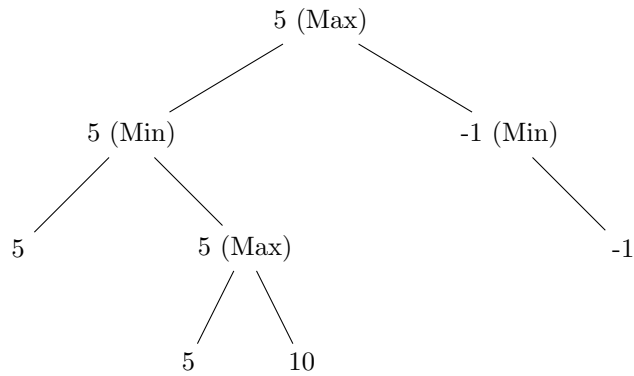
Minimax è un algoritmo utilizzato per analizzare alberi di decisione in cui esistono scelte alternative contrastanti tra due agenti o criteri. Nel caso più semplice di massimizzazione, l'algoritmo seleziona il valore massimo possibile lungo i percorsi dell'albero.

Consideriamo il seguente albero con radice Massimizzante e valori calcolati tramite Minimax:



Passaggi della valutazione Minimax:

1. Riga 1: Max
2. Riga 2: Min
3. Riga 3: Max
4. Riga 4: Min
5. $\vdots$



Per gli alberi di minimizzazione si fa:

- min
- max
- min
- max
- $\vdots$

Alpha-Beta Pruning

Alpha-Beta Pruning è un miglioramento dell'algoritmo Minimax che permette di “potare” (cioè evitare di esplorare) rami dell'albero che non possono influenzare la decisione finale. Questo riduce significativamente il numero di nodi da valutare, mantenendo invariato il risultato finale.

Durante la valutazione, si mantengono due valori:

- α : il miglior valore (massimo) che il giocatore Max può garantire finora;
- β : il miglior valore (minimo) che il giocatore Min può garantire finora.

Se in un nodo si scopre che $\beta \leq \alpha$, si può interrompere la valutazione di quel ramo (potatura), perché non influenzerà la scelta finale.

4 Alberi binari di ricerca

Sono alberi binari con la forma: $T.left.value \leq T.value \leq T.right.value$.

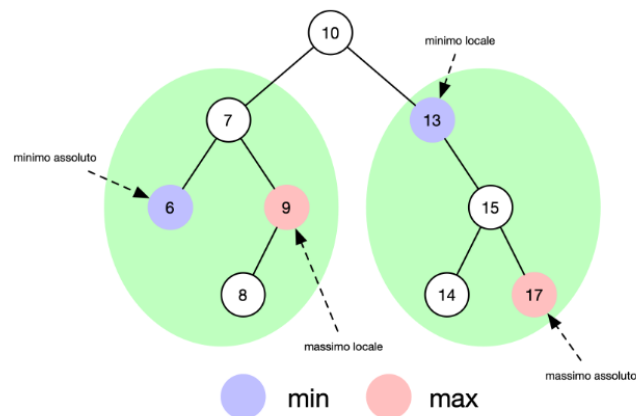
```
struct BST{
    int key;
    int value; //questo e' il tipo di dato che memorizziamo
    BST* left;
    BST* right;
    //costruttore
    BST(const int key, const int value) : key(key), value(value), left(nullptr), right(nullptr) {}
public:
    BST* insert(const int key, const int value) { ...
    }
    BST* delete_val(int key) { ...
    }
    BST* search(const int key){ ...
    }
};
```

Le funzioni elementari sono:

- `insert(key, value)` $\Rightarrow$ inserisce il nuovo nodo e lo ritorna.
- `delete(key)` $\Rightarrow$ elimina il nodo con quella value e lo ritorna.
- `search(key)` $\Rightarrow$ ritorna il nodo con quella value.

Complessita': $\Theta(n)$

Trovare il massimo e/o il minimo locale/assoluto e' molto semplice:



Infatti il nodo piu' a sinistra in basso sara' il minimo assoluto, mentre il nodo piu' a destra in basso sara' il massimo assoluto.

5 Alberi binari AVL

Sono alberi binari di ricerca quasi perfetti.

```
struct AVL {
private:
    int val;
    AVL* ltree;
    AVL* rtree;
    int bilanciamento;
public:
    AVL(int v) : val(v), ltree(nullptr), rtree(nullptr), bilanciamento(0) {}
private:
    int height() const { ...
    }
    void update_bilanciamento() { ...
    }
    AVL* rotateLeft() { ...
    }
    AVL* rotateRight() { ...
    }
    AVL* balance() { ...
    }
    AVL* minValueNode() { ...
    }
public:
    AVL* insert(int newVal) { ...
    }
    AVL* deleteNode(int key) { ...
    }
    bool search(int x) const { ...
    }
};
```

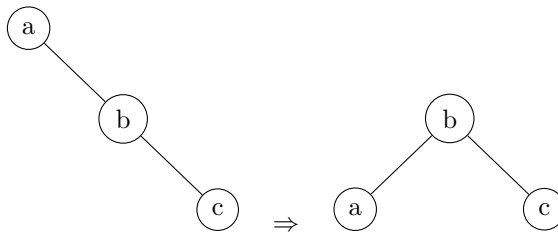
Figure 9: Versione semplificata dove $key = value$

Ovvero si calcola con indice $\beta = height(ltree) - height(rtree)$ e se $|\beta| \leq 1$ l'albero è bilanciato.

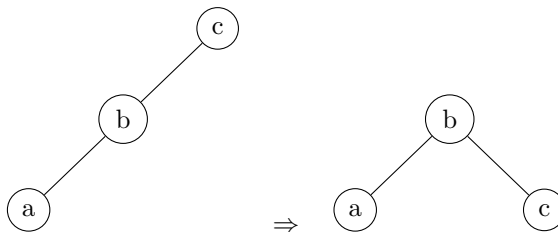
Quindi l'albero sarà sempre bilanciato in altezza (o quasi perfettamente) dandoci così dei tempi di ricerca delle funzioni di complessità: $\Theta(\log n)$.

Vediamo come l'albero si bilancia:

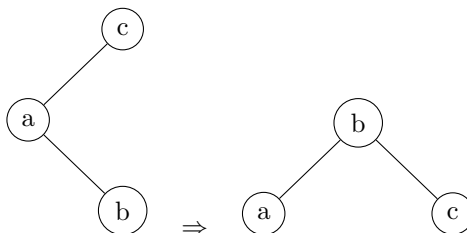
- **Rotazione a sinistra**



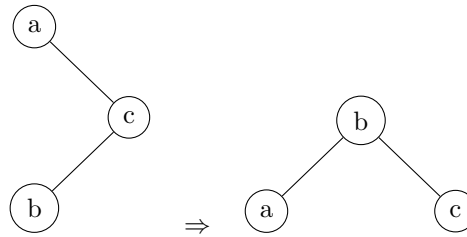
- **Rotazione a destra**



- **Rotazione sinistra-destra**



- Rotazione destra-sinistra



6 Hash Table

E' una struttura dati dizionario estremamente efficiente per le funzioniz basilari.

```

template <typename K, typename V> //tipo key & tipo data
class HashTable {
private:
    static const int TABLE_SIZE = 10;
    std::vector<std::list<std::pair<K, V>>> table;

    int hash(const K& key) const { //funzione/i di hashing...
    }

public:
    //costruttore
    HashTable() : table(TABLE_SIZE) {} //massimo TABLE_SIZE liste
    //funzioni basiche
    void insert(const K& key, const V& value) { ...
    }
    bool remove(const K& key) { ...
    }
    bool search(const K& key, V& value) const { ...
    }
};
  
```

Figure 10: Versione dove K indica il tipo di dato della *key* e dove V indica il tipo di dato per il *dato*

Per tutte le funzioni basiche abbiamo $\Theta(1)$ ($O(n)$ per insert e search) come costo computazionale.

Vediamo la regola generale per hashing basato sulla divisione:

$$\text{hash}(\text{key}) \equiv_n \text{key}$$

```

int hash1(const int key){
    //mcd(TABLE_SIZE, key)
    return key % TABLE_SIZE;
}

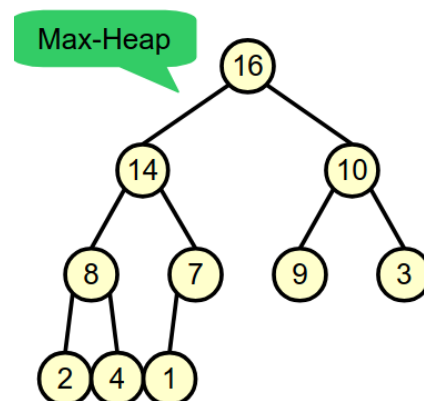
// funzione hash con probing: i è l'indice di tentativi (0, 1, 2, ...)
int hash(const int key, int i){
    //mcd(TABLE_SIZE, h1(k)+3i)
    return (hash1(key) + 3 * i) % TABLE_SIZE;
}
  
```

Ricorda che $x \equiv_n = \text{int}(x - n)$, e anche che se $x < n \rightarrow x \equiv_n (x)$.

7 Heap

BinaryHeap

E' un albero binario quasi perfetto con $T.val \geq T.child$ (*max-heap*):



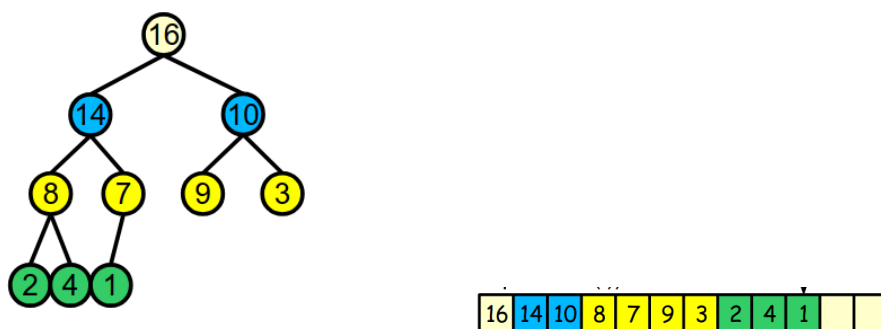
- quindi $A.val \geq A.child.val$ in caso di **max-heap**
- parallelamente abbiamo $A.val \leq A.child.val$ nel **min-heap**

Abbiamo come funzioni:

- `insert()` costo: $\Theta(\log k)$ (*Attenzione: k indica gli slot occupati dell'array*)

ArrayHeap

Possiamo descrivere un BinaryHeap come array:



Quindi nel array avremo $A[1]$ come genitore, mentre a seguire i figli con visione a livello.

- Parent: $A[i]$
- Left child: $A[2 * i]$
- Right child: $A[2 * i + 1]$

Le funzioni basi (caso max-heap) sono:

- `findMax()` $\Rightarrow A[1]$, costo di $\Theta(1)$.
- `fixHeap()` sistema la struttura per tornare ad essere un max-heap, costo $\Theta(\log n)$. (*attenzione: la sostituzione e' sempre con il figlio dal valore maggiore*)
- `heapify(int arr[])` dato una array restituisce un array-heap, costo $\Theta(n)$.
Tutorial: si rendono maxHeap (per massimizzare) i figli sx e dx, poi si controlla se $T \geq T.sx$ e se $T \geq T.dx$ e se non e' cosi' si inverte T con il maggiore, poi si applica `fixheap` sul sottoalbero con radice il vecchio T .
- `deleteMax` rimuove $A[1]$, costo $\Theta(\log n)$.
Tutorial: cancelli $A[1]$ e lo sostituisce con $A[n]$ poi fai `fixheap`.

Nota: $\log n$ indica come sempre la profondita' dell'albero.

8 Union-Find

Struttura che gestisce insiemi disgiunti.

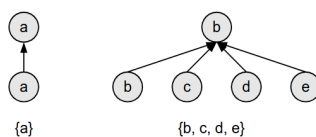
Definiamo *rappresentante* un qualsiasi elemento che ci permette di rappresentare tutto l'insieme.

```
template <typename T>
struct union_find {
private:
    std::unordered_map<T, T> parent;
public:
    // Crea un nuovo insieme contenente solo elem
    void makeset(const T& elem) { ...
    }
    // Trova il rappresentante dell'insieme contenente elem
    T find(const T& elem) { ...
    }
    // Unisce i due insiemi con rappresentanti elem1 e elem2
    void unite(const T& elem1, const T& elem2) { ...
    }
};
```

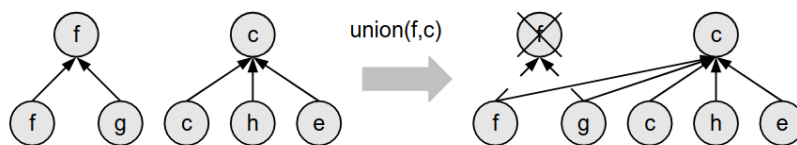
1) Possiamo descrivere una struttura union-find come un albero di altezza 1:

QuickFind: struttura con

- makeset() : $O(1)$
- find() : $O(1)$
- union() : $O(n)$



Per ottimizzare il tempo di union() possiamo unire i due insiemi con quello che ha piu' elementi come rappresentante (euristica sul peso)

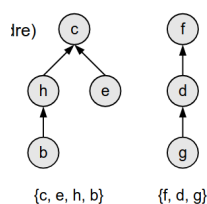


$\Rightarrow \text{union}() : O(\log n)$

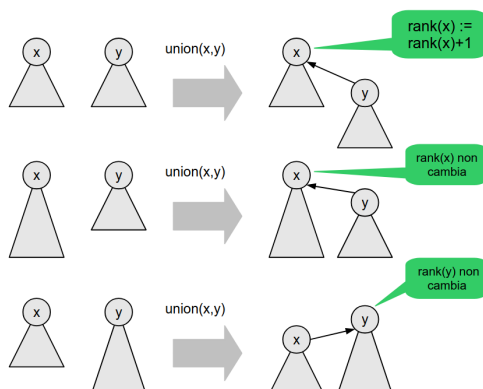
2) Possiamo anche descrivere union-find come un albero radicato generico:

QuickUnion: struttura con

- makeset() : $O(1)$
- find() : $O(n)$
- union() : $O(1)$



Per non far diventare gli alberi troppo alti rendiamo la radice dell'albero minore figlia della radice dell'albero maggiore:



$\Rightarrow \text{find}() : O(\log n)$

9 Grafi

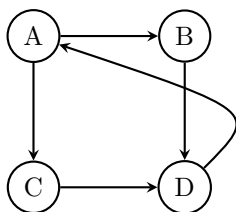
E' una struttura matematica dove si rappresentano coppie:

$$G = (V, E)$$

- V (vertici) e' l'insieme dei nodi
- E (archi) sono le connessioni tra i nodi

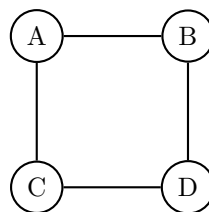
Grafo orientato

Gli archi hanno direzione



Grafo non orientato

Gli archi non hanno direzione



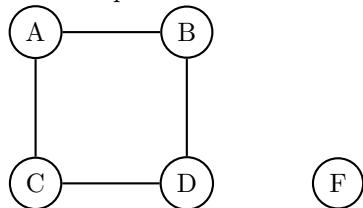
Un vertice w e' adiacente ad v se e solo se $(w, v) \in E$.

Grafo pesato

Ad ogni arco si puo' definire un peso (o costo), questo puo' essere definito tramite

$$c : E \rightarrow \mathbb{R}$$

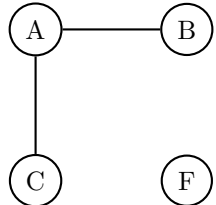
Per esempio:



- $c((A, B)) = n \quad (n \in \mathbb{R})$
- $c(A, F) = \infty$

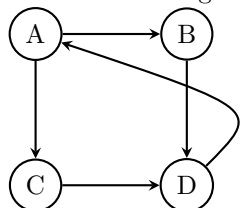
Grado

definiamolo come il numero di archi che partono da un vertice.



- A ha grado 2
- B e C hanno grado 1
- F ha grado 0

Se si lavora con grafi orientati allora definiamo grado entrante (e) e grado uscente (u).

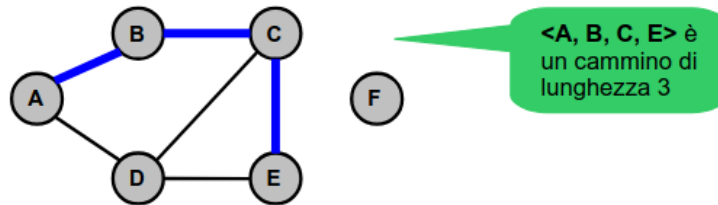


- A ha g.u 2 e ha g.e 1
- B, C hanno g.u 1 e g.e 1
- D ha g.u 1 e g.e 2

Quindi il **grado** di un vertice e' la somma dei gradi entranti e uscenti di quel vertice.

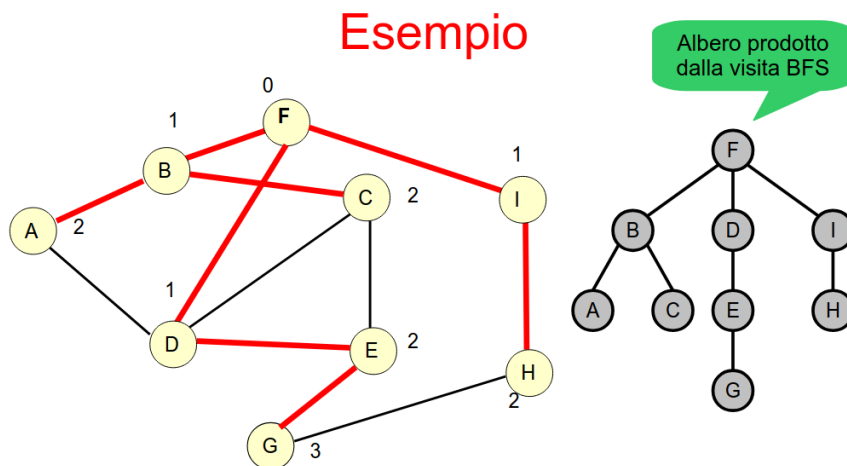
Cammini

E' una sequenza di vertici adiacenti per arrivare ad un'altro vertice:



Questo e' un cammino **semplice**. Si dice cammino **non** semplice quando dei vertici si ripetono.

Per la visione dei vertici $v \in V$ possiamo creare una similitudine tra i grafi e gli alberi tali che utilizzando la visita BFS otteniamo



Nota: i numeri sono la distanza tra il vertice radice F e le foglie vertici.

Pseudo codice per la visione BFS per i grafi:

Algorithm 1 BFS(Grafo $G = (V, E)$, vertice v)

```

1: for  $x \in V$  do
2:    $x.visit \leftarrow \text{false}$ 
3:    $x.dist \leftarrow \infty$ 
4: end for
5: Queue  $q \leftarrow \text{new Queue}()$ 
6:  $v.visit \leftarrow \text{true}$ 
7:  $v.dist \leftarrow 0$ 
8:  $q.enqueue(v)$ 
9: while not  $q.isEmpty()$  do
10:   $u \leftarrow q.dequeue()$ 
11:  for  $w \in u.adj()$  do
12:    if not  $w.visit$  then
13:       $w.dist \leftarrow u.dist + 1$ 
14:       $q.enqueue(w)$ 
15:       $w.visit \leftarrow \text{true}$ 
16:    end if
17:  end for
18: end while

```

Algorithm 2 hasCycle($G = (V, E)$)

```
1: Input: grafo orientato  $G = (V, E)$ 
2: Output: true se  $G$  contiene almeno un ciclo, false altrimenti
3: for all  $v \in V$  do
4:    $state[v] \leftarrow 0$  ▷ 0 = non visitato
5: end for
6: for all  $v \in V$  do
7:   if  $state[v] = 0$  then
8:     if DFS( $v$ ) then
9:       return true
10:    end if
11:  end if
12: end for
13: return false

14: function DFS( $u$ ) ▷ 1 = in esplorazione
15:    $state[u] \leftarrow 1$ 
16:   for all  $v$  adiacente a  $u$  do
17:     if  $state[v] = 1$  then
18:       return true ▷ ciclo trovato
19:     end if
20:     if  $state[v] = 0$  then
21:       if DFS( $v$ ) then
22:         return true
23:       end if
24:     end if
25:   end for
26:    $state[u] \leftarrow 2$  ▷ 2 = completamente esplorato
27:   return false
28: end function
```

Algorithm 3 Dijkstra(Grafo $G = (V, E, w)$, sorgente s , destinazione d)

```
1: for  $x \in V$  do
2:    $x.dist \leftarrow \infty$ 
3: end for
4:  $s.dist \leftarrow 0$ 
5: PriorityQueue  $Q \leftarrow$  new PriorityQueue()
6:  $Q.push((0, s))$  ▷ Coppia (distanza, nodo)
7: while not  $Q.isEmpty()$  do
8:    $(currDist, u) \leftarrow Q.pop\_min()$  ▷ Estrai nodo con distanza minima
9:   if  $u = d$  then
10:    return currDist ▷ Distanza minima trovata
11:  end if
12:  if  $currDist < u.dist$  then
13:    for  $v \in u.adj()$  do
14:       $alt \leftarrow u.dist + w(u, v)$ 
15:      if  $alt < v.dist$  then
16:         $v.dist \leftarrow alt$ 
17:         $Q.push((alt, v))$ 
18:      end if
19:    end for
20:  end if
21: end while
22: return  $d.dist$  ▷ Se  $d$  non raggiungibile sarà  $\infty$ 
```

10 Greedy

- Ordina gli elementi secondo un criterio (ad esempio, capacità, valore, tempo di fine).
- Scorri gli elementi ordinati e seleziona quelli che soddisfano il vincolo, scegliendo la soluzione localmente ottima in ogni passo.
- Alla fine, l'insieme selezionato sarà la soluzione ottima globale (solo se il problema soddisfa il principio di optimalità).

Algorithm 4 countDisjointIntervals(array $A[1 \dots n]$)

```
1: Input: array  $A[1 \dots n]$  di coppie  $(a_i, b_i)$ 
2: Output: numero massimo di intervalli disgiunti
3: sort  $A$  by  $A.second$  crescente
4: int  $counter \leftarrow 1$ 
5:  $(last\_start, last\_end) \leftarrow A[1]$ 
6: for  $i = 2$  to  $n$  do
7:   if  $last\_end < A[i].first$  then
8:      $(last\_start, last\_end) \leftarrow A[i]$ 
9:      $counter \leftarrow counter + 1$ 
10:  end if
11: end for
12: return  $counter$ 
```

11 Programmazione dinamica

11.1 Sottovettore di valore massimo

Problema: Dato $A[1 \dots n]$ si cerca il sottovettore con la somma degli elementi in ordine sia massima

$$\forall i = 1 \dots n$$

$$S[i] = \max \left\{ \begin{array}{l} V[i] \\ S[i-1] + V[i] \end{array} \right\}$$

Algorithm 5 sottovettoreMax(int $V[1 \dots n]$)

```
1: int  $S[1 \dots n]$ ;
2:  $S[1] = V[1]$ 
3: int  $MaxValue = S[1]$ ;
4: for  $i = 2$  to  $n$  do
5:   if  $S[i-1] + V[i] > V[i]$  then
6:      $S[i] \leftarrow S[i-1] + V[i]$ 
7:   else
8:      $S[i] \leftarrow V[i]$ 
9:   end if
10:  if  $S[i] > MaxValue$  then
11:     $MaxValue \leftarrow S[i]$ 
12:  end if
13: end for
14: return  $MaxValue$ 
```

Ritorniamo quindi la somma massima di alcuni elementi ordinati di A .

11.2 Problema dello zaino

Dati n elementi con un peso specifico e un valore specifico, cerchiamo la combinazione massima di valore degli elementi con peso totale inferiore a K .

Definiamo $V[i, K]$ l'array che simula lo zaino.

$$\begin{cases} V[i, 0] = 0 & \forall i = 1 \dots n \text{ (ovvero non c'è uno zaino)} \\ V[1, K] = v[1] & \text{se } K \geq p[1] \text{ (ovvero il primo elemento sta dentro lo zaino)} \\ V[1, K] = 0 & \text{se } p[1] > K \text{ (ovvero il primo elemento NON sta dentro lo zaino)} \end{cases}$$

Se $p[i] > K$: allora l'elemento i non può mai stare dentro lo zaino

Se $p[i] \leq K$:

- Inseriamo l'oggetto i nello zaino, $V[i, K] = V[i - 1, K - p[i]] + v[i]$
- Non inseriamo l'oggetto: $V[i, K] = V[i - 1, K]$

Dato che stiamo massimizzando:

$$V[i, K] = \max \left\{ \begin{array}{l} V[i - 1, K - p[i]] + v[i] \\ V[i - 1, K] \end{array} \right\}$$

Quindi:

$$V[i, K] = \begin{cases} V[i - 1, K] & \text{se } p[i] > K \\ \max\{V[i - 1, K - p[i]] + v[i], V[i - 1, K]\} & \text{se } p[i] \leq K \end{cases}$$

Tabella di programmazione dinamica / matrice V

$$\begin{array}{lcl} p = & [& 2, \quad 7, \quad 6, \quad 4 \\ v = & [& 12.7, 6.4, \quad 1.7, \quad 0.3] \end{array}$$

		j →										
		0	1	2	3	4	5	6	7	8	9	10
↓ i	1	0.0	0.0	12.7	12.7	12.7	12.7	12.7	12.7	12.7	12.7	12.7
	2	0.0	0.0	12.7	12.7	12.7	12.7	12.7	12.7	12.7	19.1	19.1
	3	0.0	0.0	12.7	12.7	12.7	12.7	12.7	12.7	14.4	19.1	19.1
	4	0.0	0.0	12.7	12.7	12.7	12.7	13.0	13.0	14.4	19.1	19.1

Algorithm 6 ZainoIntero($v[1..n]$, $p[1..n]$, K)

```

1: int  $V[0..n][0..K]$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 0$  to  $K$  do
4:     if  $p[i] > j$  then
5:        $V[i][j] \leftarrow V[i - 1][j]$ 
6:     else
7:        $V[i][j] \leftarrow \max(V[i - 1][j], V[i - 1][j - p[i]] + v[i])$ 
8:     end if
9:   end for
10: end for
11: //stampiamo gli indici
12: int  $j \leftarrow K$ 
13: for  $i = n$  downto 1 do
14:   if  $p[i] \leq j$  and  $V[i][j] = V[i - 1][j - p[i]] + v[i]$  then
15:     print( $i$ )
16:      $j \leftarrow j - p[i]$ 
17:   end if
18: end for
19: return  $S$ 

```
